

NOSHASHI

AUTONOMOUS COMPLIANCE LAYER

Compliance on the XRP Ledger is a report written after the fact.
NOSHASHI makes it infrastructure that runs before the signature.

INFRASTRUCTURE BRIEFING | VERSION 0.1.0 | 21 AUGUST 2026

01 / THE OBJECTIVE

Make regulated value move without anyone guessing whether it is allowed to

Every regulated transfer on a public ledger faces the same question: is this counterparty eligible, under this rule set, right now? Today that question is answered after settlement, by reconciling logs. The first signal that a transaction was ineligible is that it failed, was reversed, or surfaced in an examination months later.

NOSHASHI moves the answer to before the signature. A settlement is described, evaluated against the destination Permissioned Domain's published rule set, and answered GO, HOLD or NO-GO together with the exact rule that decided it. The verdict is hashed into a receipt that an auditor can verify without ever seeing the identity behind it.

Nothing in the evaluation path touches the ledger. The gate runs entirely on read operations, which is precisely why it can run before a transaction exists.

THE THREE-STEP MECHANISM

STEP	WHAT HAPPENS	WHY IT MATTERS
1 DESCRIBE	Destination, amount and target permissioned domain are named.	No signature, no broadcast, no fee at risk.
2 ADJUDICATE	Credentials, reserve, ceiling and governance are evaluated in a fixed order.	Deterministic: identical inputs always produce an identical verdict.
3 RECEIPT	A canonical SHA-256 digest of the verdict body is produced.	Provable that the check ran, without disclosing what was checked.

THE RULE SET, IN EVALUATION ORDER

A blocking failure produces NO-GO. All blocking rules passing with a failed advisory rule produces HOLD. Everything passing produces GO.

RULE	SEVERITY	TESTS
ACCOUNT_ACTIVATED	Blocking	The account exists on the validated ledger with a sequence number.
CREDENTIAL_<TYPE>	Blocking	One rule per credential the domain requires; satisfied only by an accepted, unrevoked XLS-70 object.
RESERVE_SOLVENCY	Blocking	Balance covers 1 XRP base reserve plus 0.2 XRP per owned object.
SPENDABLE_BALANCE	Blocking	The transfer fits inside balance minus reserve.
TRANSFER_CEILING	Blocking	The transfer is within the domain's per-settlement cap.
DOMAIN_GOVERNANCE	Blocking / advisory	Blocking when the domain is suspended, advisory when under review.
DOMAIN_ATTESTATION	Advisory	The account publishes a Domain field.

Four properties nothing else in this ecosystem combines

XLS-70 credentials and XLS-80 permissioned domains are young standards. There is, at the time of writing, no production console that reads both from mainnet and enforces a policy against them. That is the opening this occupies.

PRE-SETTLEMENT ADJUDICATION

Wallets tell you a transaction failed. Explorers tell you it failed. NOSHASHI tells you it would fail, before you sign, and names the rule. The distinction is the difference between a receipt and a decision.

EXPLAINABILITY AS THE PRODUCT

A boolean is worthless to a compliance officer, who has to justify a decision to a regulator. Every verdict carries an itemised trace: which rule, what threshold, what the actual value was. "NO-GO" is never the answer; "reserve requirement is 76.6 XRP for 378 owned objects" is.

VERIFIABLE WITHOUT DISCLOSURE

The receipt is a SHA-256 digest over a canonical body: verdict, domain, subject, amount, timestamp and the pass/fail of every rule. It proves a specific check ran against a specific rule set at a specific moment. It discloses nothing about the identity behind the credentials. Compliance becomes auditable without becoming public.

ZERO-EGRESS INTELLIGENCE

Every compliance copilot on the market ships your transaction context to a vendor's API, which is itself a data-handling event a regulated firm must disclose. The NOSHASHI agent defaults to a model running on the operator's own machine. It can see wallet addresses, receipts and rule traces precisely because none of it crosses the network boundary.

THE FORWARD BETS

Specified, sequenced, and not yet claimed as shipped:

CONCEPT	THE GAP IT FILLS
Proof-of-Check	Compliance checks are private logs. Anchoring the receipt digest on-ledger makes one provable without trusting the party that ran it.
Policy Drift Radar	When a domain tightens its requirements, members find out by being rejected mid-transaction. Recompute eligibility the moment the rule set moves.
Credential Expiry Radar	XLS-70 credentials expire silently. A wallet compliant on Monday is locked out Tuesday with no warning.
Counterparty Risk Graph	Sanctions screening stops at the direct counterparty. Second-hop exposure is invisible.
Portable Compliance Passport	Every venue re-runs the same KYC. A revocable bundle of zero-knowledge predicates carries between them.
Verification SLA Oracle	Institutions cannot contract against infrastructure whose uptime nobody publishes.

03 / INFRASTRUCTURE

What the system is actually made of

Five layers. Every one is running; none is a mock.

LAYER 1 / DESKTOP CLIENT

COMPONENT	IMPLEMENTATION
Shell	Tauri v2 (Rust) with a WKWebView front end. Native macOS bundle, 2.4 MB DMG.
Interface	React 18, TypeScript strict, Vite, Tailwind v4, Radix primitives, Framer Motion.
Windows	Frameless console (1280x820) plus a 380x560 menu bar HUD anchored to the tray icon.
Menu bar	Live ticker showing gate state and ledger height, global accelerator (Cmd+Shift+X), native menu, notifications.
Offline	Typefaces self-hosted; no font CDN, no third-party origin, works without a network for everything but ledger reads.

LAYER 2 / LEDGER TRANSPORT

COMPONENT	IMPLEMENTATION
Connection	One persistent WebSocket to XRPL mainnet, shared by every read in the app.
Failover	Three public endpoints, exponential backoff, automatic rotation on close.
Why WebSocket	Public rippled HTTP endpoints send no CORS headers, so a web view cannot POST to them at all. The WebSocket API is not subject to CORS and exposes the same command set.
Live streams	Ledger closes and validated transactions, driving the strip charts and the ticker.
Reads	account_info, account_objects (credentials), account_tx, server_info, fee, ledger.

LAYER 3 / POLICY ENGINE

A pure, synchronous, deterministic evaluator. It is implemented twice on purpose: once in the client for instant feedback, once server-side for enforcement, as a byte-for-byte mirror so a verdict can never differ between the console and the API. Receipts are SHA-256 over a canonical JSON body, so both sides produce an identical digest.

LAYER 4 / BACKEND

COMPONENT	IMPLEMENTATION
Database	Supabase Postgres. Dedicated noshashi schema, 9 tables, row level security on every one, scoped to auth.uid().
Identity	Supabase Auth. PKCE, bcrypt-hashed passwords, TOTP two-factor (RFC 6238), email one-time codes.
Entitlements	A single derived row per account, writable only by the Stripe webhook under the service role. A client cannot grant itself a plan.
Edge Functions	noshashi-checkout, noshashi-portal, noshashi-stripe-webhook, noshashi-verify (the Compliance API).
Compliance API	POST /v1/verify. API-key auth (SHA-256 hashed at rest), atomic prepaid-credit decrement in SQL, per-call audit row, 60 req/min limiter.

LAYER 5 / INTELLIGENCE

COMPONENT	IMPLEMENTATION
Default	Local runtime. Ollama, LM Studio, llama.cpp, Jan and vLLM are auto-detected. Free, and no prompt leaves the machine.
Optional	Claude, OpenAI, Groq, OpenRouter, DeepSeek, Mistral or any OpenAI-compatible endpoint. Keys held in the OS keyring, one entry per provider, TLS enforced.
Grounding	The system prompt carries live ledger state and the real rule set. The agent explains verdicts; it never issues them.
Support brain	A deterministic knowledge base plus self-healing diagnostics that run with no AI, no account and no network. The model is an upgrade, never a dependency.

04 / SECURITY AND DATA

The safest data is data nobody holds

Without an account the application has no server-side state whatsoever. It reads public ledger data, keeps preferences in a local file, and never identifies the user to anything we operate. There is no analytics vendor, no telemetry, no crash reporting and no advertising identifier in the build. Signing in adds exactly the data enumerated in the privacy policy and nothing beyond it.

CONTROLS IN PLACE

CONTROL	IMPLEMENTATION
Content Security Policy	Strict allow-list. No unsafe-eval, object-src none, frame-ancestors none. Every outbound host named explicitly.
Bridge exposure	withGlobalTauri disabled; the web view has no ambient bridge to invoke native commands.
Native surface	No filesystem or shell plugin. Two narrow commands, each validating scheme, path separators and size.
Isolation	Row level security on every table; entitlements and subscriptions are read-only to clients.
Payments	No card detail reaches the application. Price IDs validated server-side against an allow-list; seat counts clamped.
Webhook trust	Stripe signature verified in constant time with a five-minute replay window.
API keys	256-bit CSPRNG, stored only as a SHA-256 digest, displayed exactly once.
Dependencies	Zero production vulnerabilities; no GPL, AGPL or SSPL components in the tree.

KNOWN GAPS

Named rather than hidden, because a security page that lists only strengths is a marketing page:

- The build is **not code-signed or notarized**. macOS Gatekeeper warns on first launch until an Apple Developer account is attached.
- **style-src still permits unsafe-inline**, required by the styling and animation layers. Removing it needs a nonce-based build pipeline.
- **Edge Functions carry only a per-instance rate limiter**. A managed limiter is required before the API is opened to third parties.
- **Model-provider keys pass through the web view** at call time. Local runtimes, the default, need no key at all.
- **Leaked-password protection is not yet enabled** in the auth provider. It is a single dashboard toggle.
- **No penetration test has been performed**. One engagement is required before taking institutional money.

05 / COMMERCIAL MODEL

Free where a human decides, metered where a machine does

The entire console is free forever, with no account required: live mainnet telemetry, unlimited gate checks, the credential registry, the domain grid, the audit trail with CSV export, the on-device agent and the menu bar HUD. Paid tiers begin where a desk needs something an individual does not.

TIER	PRICE	WHAT IT ADDS
OPERATOR	Free	The full console, forever. No account, no payment method, no expiry.
DESK	\$749 / seat / mo	Multi-wallet portfolios, policy drift and credential expiry alerting, receipt vault with export, 5,000 API verifications, priority support.
INSTITUTION	\$4,000 / mo	Everything in Desk plus the Compliance API with keys and webhooks, white-labelled wallet, regulator read-only seats, 100,000 verifications, published SLA.

METERED VERIFICATION

Programmatic verifications through the Compliance API draw from a prepaid balance. Checks a human runs in the console are never metered: the fee attaches to automation, not to people.

PACK	PRICE	UNIT
10,000 verifications	\$450	\$0.045 each
50,000 verifications	\$2,000	\$0.040 each
250,000 verifications	\$8,750	\$0.035 each

WHAT IS LIVE TODAY

CAPABILITY	STATE
Live mainnet telemetry with failover	Live
Explainable settlement gate and receipts	Live
XLS-70 credential registry	Live
XLS-80 domain grid	Live
Exportable audit trail	Live
Multi-wallet portfolios and compliance radar	Live (Desk)
Compliance API with prepaid credits	Live (Institution)
On-device agent and offline support brain	Live
macOS menu bar residency with live ticker	Live
Stripe checkout, portal and entitlement webhook	Live, pending secrets
On-ledger receipt anchoring	Specified, in flight
Outbound webhooks and regulator seats	Specified, in flight

Why this matters now

XLS-70 and XLS-80 give the XRP Ledger the two primitives regulated finance has been waiting for: a credential that an issuer can attest and a subject can accept, and a domain that can require one. The standards exist. What does not yet exist is the layer that enforces them at the moment of settlement and produces evidence afterwards.

Whoever builds the reference implementation of that layer becomes the default answer in every integration conversation about these standards. That is the position this project is built to take, and the reason its scope is deliberately narrow: it does not custody assets, hold keys, sign transactions, or move funds. It reads, it rules, and it proves. A system that cannot lose your money is a system a regulated institution can adopt quickly.

OPERATING PRINCIPLES

- **Mainnet only.** No testnet path exists in the build. Compliance software that demos on a testnet has proven nothing.
- **Explain, never decide.** The deterministic engine adjudicates; the language model only ever explains what it decided.
- **Free where a human decides.** Paywalling a compliance check a person runs by hand would make the ecosystem worse.
- **Name the gaps.** Every unshipped capability is labelled as such, in the product and in this document.

STANDING DISCLAIMER

Nothing produced by this software, and nothing in this document, is legal, regulatory, tax or investment advice. A GO verdict means the configured rules passed. It is not a representation that a transaction is lawful in any jurisdiction and it does not discharge any obligation owed to a regulator. Determinations carrying legal consequence must be reviewed by qualified counsel and the responsible compliance officer.

support@noshashi.app · institutions@noshashi.app · security@noshashi.app

© 2026 NOSHASHI Labs. All rights reserved.